Problem Statement:

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

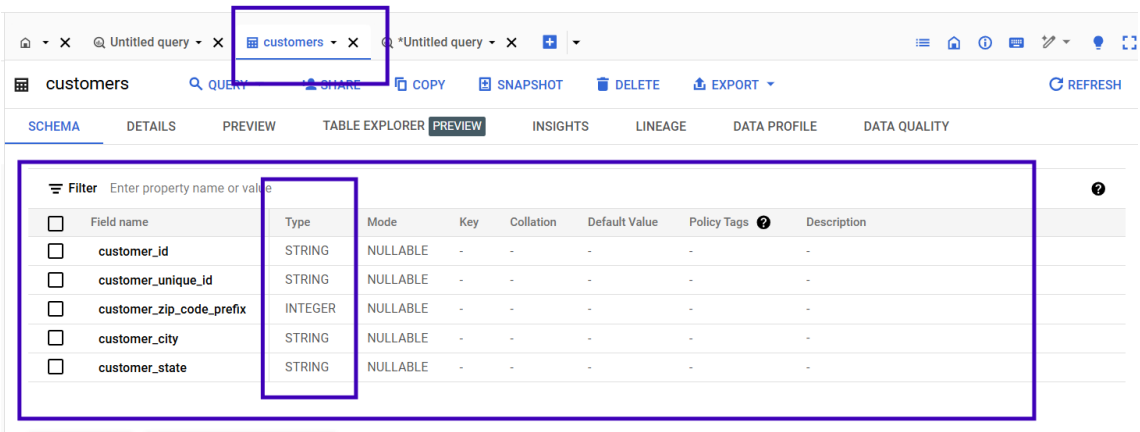
What does 'good' look like?

1)Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

Question: -1. Data type of all columns in the "customers" table.

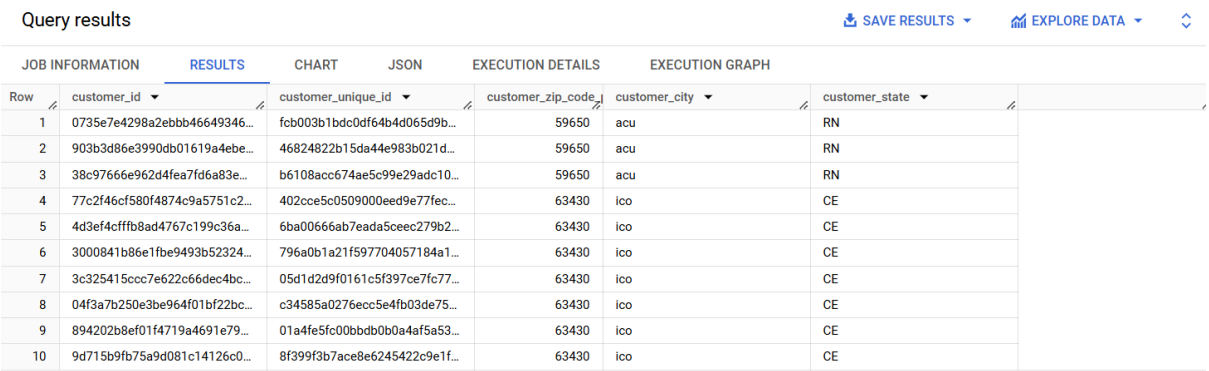
Query:-

```
SELECT * FROM `scaler-dsml-sql-priyanka.Target_SQL.customers` LIMIT 10
```



Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
customer_id	STRING	NULLABLE	-	-	-	-	-
customer_unique_id	STRING	NULLABLE	-	-	-	-	-
customer_zip_code_prefix	INTEGER	NULLABLE	-	-	-	-	-
customer_city	STRING	NULLABLE	-	-	-	-	-
customer_state	STRING	NULLABLE	-	-	-	-	-

Query Output: -



Row	customer_id	customer_unique_id	customer_zip_code_p	customer_city	customer_state
1	0735e7e4298a2ebbb46649346...	fcb003b1bdc0df64b4d065d9b...	59650	acu	RN
2	903b3d86e3990db01619a4ebe...	46824822b15da44e983b021d...	59650	acu	RN
3	38c97666e962d4fea7fd6a83e...	b6108acc674ae5c99e29adc10...	59650	acu	RN
4	77c2f46cf580f4874c9a5751c2...	402cce5c0509000eed9e77fec...	63430	ico	CE
5	4d3ef4cfffb8ad4767c199c36a...	6ba00666ab7eada5ceec279b2...	63430	ico	CE
6	3000841b86e1fbe9493b52324...	796a0b1a21f597704057184a1...	63430	ico	CE
7	3c325415ccc7e622c66dec4bc...	05d1d2d9f0161c5f397ce7fc77...	63430	ico	CE
8	04f3a7b250e3be964f01bf22bc...	c34585a0276ecc5e4fb03de75...	63430	ico	CE
9	894202b8ef01f4719a4691e79...	01a4fe5fc00bbdb0b0a4af5a53...	63430	ico	CE
10	9d715b9fb75a9d081c14126c0...	8f399f3b7ace8e6245422c9e1f...	63430	ico	CE

Insights:- The data consists of STRING type fields for customer identification and location details, and an INTEGER type for the zip code prefix.

Question: -2) Get the time range between which the orders were placed.

Query:-

```
SELECT

extract(Year from order_purchase_timestamp)as Year,

Min(order_purchase_timestamp)as Min_timestamp,

Max(order_purchase_timestamp)as Max_timestamp,

Timestamp_diff(Max(order_purchase_timestamp),Min(order_purchase_timestamp),DAY)as Days_diff

FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`

GROUP BY Year

Order by Year
```

Query Output: -

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	Year	Min_timestamp	Max_timestamp	Days_diff
1	2016	2016-09-04 21:15:19 UTC	2016-12-23 23:16:47 UTC	110
2	2017	2017-01-05 11:56:06 UTC	2017-12-31 23:29:31 UTC	360
3	2018	2018-01-01 02:48:41 UTC	2018-10-17 17:30:18 UTC	289

Insights:-

Time range between which the orders were placed were Sep 2016 (Min Timestamp) to Oct 2018(Max timestamp).

The data shows the time span (in days) between the earliest and latest orders for each year.

For example, in 2016, orders were placed over a span of 110 days, while in 2017, the span was 360 days & IN 2018 the span was 289 Days.

3)Count the Cities & States of customers who ordered during the given period.

Query:-

```
SELECT

count(Distinct customer_city)as City,count(Distinct customer_state)as state

FROM

`scaler-dsml-sql-priyanka.Target_SQL.customers`as C

Inner JOIN

`scaler-dsml-sql-priyanka.Target_SQL.orders`as O

ON

C.customer_id = O.customer_id
```

Query Output: -

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	City	state
1	4119	27

Insights:-

The data shows that customers who placed orders are spread across **4,119 distinct cities** and **27 distinct states,** Who ordered during the given period.

2.In-depth Exploration:

1.Is there a growing trend in the no. of orders placed over the past years?

Query:-

```
SELECT extract(year from order_purchase_timestamp)as order_year,

Count(DISTINCT order_id)as total_orders,

FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`

group by order_year

order by order_year ASC
```

Query Output: -

Press Alt+F1 for Accessibility Options.

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	order_year	total_orders
1	2016	329
2	2017	45101
3	2018	54011

Insights:-

The data shows that the total number of orders for each year, ordered by order_year in ascending order. The growth between the years is as follows:

- From 2016 to 2017, the number of orders increased by **44,772.**
- From 2017 to 2018, the number of orders increased by **8,910**

2.Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Query:-

```
SELECT

EXTRACT(YEAR FROM order_purchase_timestamp) AS year,

CASE

WHEN EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 3 THEN 'Summer'

WHEN EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 4 AND 6 THEN 'Winter'

WHEN EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 7 AND 9 THEN 'Rain'

WHEN EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 10 AND 12 THEN 'Autumn'
```

```
ELSE 'Unknown'

END AS Brazil_season_orders,

COUNT(DISTINCT order_id) AS order_count

FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`

GROUP BY year, Brazil_season_orders

ORDER BY order_count DESC
```

Query Output: -

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	year	Brazil_season_orders	order_count
1	2018	Summer	21208
2	2018	Winter	19979
3	2017	Autumn	17848
4	2018	Rain	12820
5	2017	Rain	12642
6	2017	Winter	9349
7	2017	Summer	5262
8	2016	Autumn	325
9	2016	Rain	4
10	2018	Autumn	4

Insights:- conclusion is that as per Brazilian season mostly order placed BETWEEN 1 AND 3 months of 2018 duration which is summer in Brazil.

3.During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon

Query:-

```
select

CASE

When extract(hour from order_purchase_timestamp)between 0 AND 6 Then 'Dawn'

When extract(hour from order_purchase_timestamp)between 7 AND 12 Then 'Mornings'

When extract(hour from order_purchase_timestamp)between 13 AND 18 Then 'Afternoon'

When extract(hour from order_purchase_timestamp)between 19 AND 23 Then 'Night'

Else'Not Found'

End AS day_time_period,

Count(Distinct order_id)as total_orders

FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`
```

```
Group by day_time_period

Order by total_orders DESC
```

Query Output: -

Query results

JOB INFORMATION | RESULTS | CHART | JSON

Row	day_time_period	total_orders
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242

Insights:- conclusion is that Brazilian customers mostly place their orders d During Afternoon as in the afternoon total number of placed orders are 38135 highest then another time_period of day.

3.Evolution of E-commerce orders in the Brazil region:

1. **Get the month on month no. of orders placed in each state.**

Query:-

```
SELECT C.customer_state AS state,
EXTRACT(YEAR FROM O.order_purchase_timestamp) AS Order_year,
EXTRACT(MONTH FROM O.order_purchase_timestamp) AS order_month,
COUNT(DISTINCT O.order_id) AS no_of_orders
FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`AS O
JOIN `scaler-dsml-sql-priyanka.Target_SQL.customers` AS C
ON C.customer_id = O.customer_id
Group by Order_year, order_month,state
Order by no_of_orders,Order_year,order_month ASC,state ASC
```

Query Output: -

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	state	Order_year	order_month	no_of_orders
1	RR	2016	9	1
2	RS	2016	9	1
3	PB	2016	10	1
4	PI	2016	10	1
5	RR	2016	10	1
6	PR	2016	12	1
7	MS	2017	1	1
8	AM	2017	6	1
9	AP	2017	7	1
10	RR	2017	7	1

Results per page: 50 1 – 50 of 565

Insights: - conclusion is that this Query provides the month-on-month number of orders placed in each state.

2.How are the customers distributed across all the states?

Query:-

```
SELECT customer_state,

Count(distinct customer_id)as Customer_total_Count

FROM `scaler-dsml-sql-priyanka.Target_SQL.customers`

Group by customer_state

Order by Customer_total_Count desc
```

Query Output: -

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	customer_state	Customer_total
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020
11	PE	1652

Results per page: 50 1 – 27 of 27

Insights:- conclusion is that this Query provides the Total Customer Count Per State.

4.Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).
   You can use the "payment_value" column in the payments table to get the cost of orders.

Query:-

```
WITH Amounts AS(

select

Sum(Case

When EXTRACT(year from O.order_purchase_timestamp)=2018 AND EXTRACT(month FROM
O.order_purchase_timestamp) BETWEEN 1 AND 8

then P.payment_value else 0 END)as Total_Amount_2018,

Sum(Case

When EXTRACT(year from O.order_purchase_timestamp)=2017 AND EXTRACT(month FROM
O.order_purchase_timestamp) BETWEEN 1 AND 8

then P.payment_value else 0 END)as Total_Amount_2017

FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`As O

Join `scaler-dsml-sql-priyanka.Target_SQL.payments`As P

ON O.order_id = P.order_id

)

select Total_Amount_2018,Total_Amount_2017,

(Total_Amount_2018-Total_Amount_2017)/coalesce(Total_Amount_2017,1)*100 as
Percentage_increase_order_cost

from Amounts
```

Query Output: -

Query results

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	Total_Amount_2018	Total_Amount_2017	Percentage_increase_order_cost
1	8694733.839999...	3669022.119999...	136.97687164666226

Insights: - conclusion is that this Query provides **136.98% Percentage Increase** in order cost from 2017 to 2018.

- **Total_Amount_2018 = 8,694,733.84**
- **Total_Amount_2017 = 3,669,022.12**
- **Percentage Increase = 136.98%**
-

2.Calculate the Total & Average value of order price for each state.

Query:-

```
select C.customer_state,
Sum(OI.price)as total_Order_Price,
AVG(OI.price)as Avg_Order_Price
FROM `scaler-dsml-sql-priyanka.Target_SQL.customers`AS C
JOIN `scaler-dsml-sql-priyanka.Target_SQL.orders`AS O
ON C.customer_id = O.customer_id
JOIN `scaler-dsml-sql-priyanka.Target_SQL.order_items`AS OI
ON OI.order_id = O.order_id
Group by C.customer_state
Order by total_Order_Price DESC,Avg_Order_Price DESC
```

Query Output: -

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	customer_state	total_Order_Price	Avg_Order_Price
1	SP	5202955.050001...	109.6536291597...
2	RJ	1824092.669999...	125.1178180945...
3	MG	1585308.029999...	120.7485741488...
4	RS	750304.0200000...	120.3374530874...
5	PR	683083.7600000...	119.0041393728...
6	SC	520553.3400000...	124.6535775862...
7	BA	511349.9900000...	134.6012082126...
8	DF	302603.9399999...	125.7705486284...
9	GO	294591.9499999...	126.2717316759...
10	ES	275037.3099999...	121.9137012411...
11	PE	262788.0299999	145.5083222591

Results per page: 50 1 – 27 of 27

Insights: - conclusion is that this Query provides the total and average order prices for each customer state, ordered by the total order price first (highest to lowest) and then by the average order price (highest to lowest).

3.Calculate the Total & Average value of order freight for each state.

Query:-

```
select C.customer_state,
Sum(OI.freight_value)as total_freight_value,
AVG(OI.freight_value)as Avg_freight_value
FROM `scaler-dsml-sql-priyanka.Target_SQL.customers`AS C
```

```
JOIN `scaler-dsml-sql-priyanka.Target_SQL.orders`AS O
ON C.customer_id = O.customer_id
JOIN `scaler-dsml-sql-priyanka.Target_SQL.order_items`AS OI
ON OI.order_id = O.order_id
Group by C.customer_state
Order by total_freight_value DESC,Avg_freight_value DESC
```

Query Output: -

Query results

SAVE RESULTS EXPLO

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	customer_state	total_freight_value	Avg_freight_value
1	SP	718723.06999999518	15.147275390419265
2	RJ	305589.31000000093	20.960923931682579
3	MG	270853.46000000188	20.63016680630664
4	RS	135522.74000000229	21.735804330392845
5	PR	117851.68000000092	20.531651567944319
6	BA	100156.67999999858	26.363958936562188
7	SC	89660.260000000446	21.470368773946355
8	PE	59449.659999999873	32.917862679955654
9	GO	53114.979999999829	22.766815259322811
10	DF	50625.49999999984	21.041354945968457
11	ES	49764.599999999831	22.058776595744643
12	CE	48351.589999999887	32.714201623816017

Results per page: 50 1 – 27 of 27

Insights: - conclusion is that this Query provides the total and average order freight for each customer state, ordered by the total freight value first (highest to lowest) and then by the average freight value (highest to lowest).

5.Analysis based on sales, freight and delivery time.

1.Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Query:-

```
SELECT
FORMAT_DATE('%y-%m-%d', order_purchase_timestamp)as Order_Purchse_date,
FORMAT_DATE('%y-%m-%d',order_delivered_customer_date)as Order_Delivered_date,
DATE_DIFF(Date(order_delivered_customer_date),Date(order_purchase_timestamp), DAY) AS
        days_taken_orderdeliver
FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`
```

```
where order_purchase_timestamp IS NOT NULL

AND order_delivered_customer_date IS NOT NULL

Order by order_purchase_timestamp ASC,order_delivered_customer_date ASC

Limit 10
```

Query Output: -

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	Order_Purchse_date	Order_Delivered_date	days_taken_orderdeliver
1	16-09-15	16-11-09	55
2	16-10-03	16-10-26	23
3	16-10-03	16-10-27	24
4	16-10-03	16-11-08	36
5	16-10-03	16-11-03	31
6	16-10-03	16-10-31	28
7	16-10-03	16-10-14	11
8	16-10-03	16-11-03	31
9	16-10-03	16-11-01	29
10	16-10-04	16-10-22	18

Results per page: 50 1 – 10 of 10

Insights: - conclusion is that this Query provides the days difference between Order purchase date & order delivery date.

ii)Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Query:-

```
SELECT

FORMAT_DATE('%y-%m-%d', order_purchase_timestamp)as Order_Purchse_date,

FORMAT_DATE('%y-%m-%d',order_delivered_customer_date)as Order_Delivered_date,

DATE_DIFF(Date(order_delivered_customer_date),Date(order_purchase_timestamp), DAY) AS
        days_taken_orderdeliver,

FORMAT_DATE('%y-%m-%d',order_estimated_delivery_date)as estimated_delivery_date,

DATE_DIFF(Date(order_delivered_customer_date),Date(order_estimated_delivery_date), DAY) AS
        estimated_actual_delivery_datediff

FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`
```

```
where order_purchase_timestamp IS NOT NULL
AND order_delivered_customer_date IS NOT NULL
AND order_estimated_delivery_date IS NOT NULL
Order by order_purchase_timestamp ASC,order_delivered_customer_date ASC
Limit 10
```

Query Output: -

Query results

SAVE RESULTS EXPLORE DA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	Order_Purchse_date	Order_Delivered_date	days_taken_orderdeliver	estimated_delivery_date	estimated_actual_delivery_datediff
1	16-09-15	16-11-09	55	6-10-04	36
2	16-10-03	16-10-26	23	6-10-27	-1
3	16-10-03	16-10-27	24	6-11-07	-11
4	16-10-03	16-11-08	36	6-11-25	-17
5	16-10-03	16-11-03	31	6-11-29	-26
6	16-10-03	16-10-31	28	6-11-23	-23
7	16-10-03	16-10-14	11	6-11-23	-40
8	16-10-03	16-11-03	31	6-12-01	-28
9	16-10-03	16-11-01	29	6-11-25	-24
10	16-10-04	16-10-22	18	6-11-24	-33

Insights: - conclusion is that this Query provides the days difference between Actual Order delivery & order estimate delivery date.as I noted

As I noticed that Row 2 shows -1, meaning the order was delivered 1 day earlier than the estimate Order delivery date.

2)Find out the top 5 states with the highest & lowest average freight value.

Query:-

```
 WITH  Avg_freight AS
(SELECT S.seller_state,
ROUND(AVG(O.freight_value), 2) AS Avg_freight_value
 FROM `scaler-dsml-sql-priyanka.Target_SQL.sellers` AS S
 Join `scaler-dsml-sql-priyanka.Target_SQL.order_items` AS O
 ON S.seller_id = O.seller_id
 Group by S.seller_state
)
select Avg_freight_value,seller_state,
CASE
when RANK()over(order by Avg_freight_value ASC)<=5 then'Lowest'
```

```
when RANK()over(order by Avg_freight_value ASC)>=5 then'Highest'

Else'Not Found' End as Avg_freight_value_Ranking

From Avg_freight

Group by Avg_freight_value,seller_state

Order by Avg_freight_value ASC

Limit 10
```

Query Output: -

Query results

SAVE RESULTS EXPLORE

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	Avg_freight_value	seller_state	Avg_freight_value_Ranking
1	18.45	SP	Lowest
2	19.39	PA	Lowest
3	19.47	RJ	Lowest
4	20.57	DF	Lowest
5	22.72	PR	Lowest
6	23.29	RN	Highest
7	23.98	MS	Highest
8	24.08	MG	Highest
9	24.16	GO	Highest
10	26.03	RS	Highest

Results per page: 50 1 – 10 of 10

Insights: - conclusion is that this Query provides top 5 states with the highest & lowest average freight value by using Rank This gives the **highest** freight values the rank of 1 (and so on) & gives the **lowest** freight values the rank of 1 (and so on).

3,Find out the top 5 states with the highest & lowest average delivery time.

Query:-

```
WITH  Avg_delivery_time AS

(SELECT S.seller_state,

ROUND(AVG(DATE_DIFF(O.order_delivered_customer_date, OI.shipping_limit_date, DAY)),
2) AS average_delivery_time,

FROM `scaler-dsml-sql-priyanka.Target_SQL.order_items`as OI

JOIN `scaler-dsml-sql-priyanka.Target_SQL.orders`AS O

ON OI.order_id = O.order_id

Join `scaler-dsml-sql-priyanka.Target_SQL.sellers`AS S

ON S.seller_id =OI.seller_id

    WHERE

        O.order_delivered_customer_date IS NOT NULL
```

```
        AND OI.shipping_limit_date IS NOT NULL
Group by S.seller_state)
Select average_delivery_time,seller_state,
Case
When Rank()Over(Order by average_delivery_time ASC)<=5 THEN 'Lowest'
When Rank()Over(Order by average_delivery_time ASC)>=5 THEN 'Highest'
Else 'Not Found' End as average_delivery_time_Ranking
From Avg_delivery_time
Group by average_delivery_time,seller_state
Order by  average_delivery_time_Ranking Desc
LIMIT 10;
```

Query Output: -

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	average_delivery_time	seller_state	average_delivery_time_Ranking
1	5.45	SP	Lowest
2	5.44	MS	Lowest
3	4.41	RS	Lowest
4	5.56	RJ	Lowest
5	5.39	MG	Lowest
6	6.57	PB	Highest
7	5.86	RN	Highest
8	11.98	CE	Highest
9	6.57	GO	Highest
10	10.0	RO	Highest

Results per page: 50 1 – 10 of 10

Insights: - conclusion is that this Query provides top 5 states with the highest & lowest average delivery time by using Rank This gives the highest rank of 1 (and so on) lowest average delivery time with values the rank of 1 (and so on).

4.Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Query:-

```
WITH  State_delivery_Speed AS
(SELECT S.seller_state,
ROUND(AVG(DATE_DIFF(O.order_delivered_customer_date, O.order_estimated_delivery_date, DAY)),
2) AS delivery_Speed,
FROM `scaler-dsml-sql-priyanka.Target_SQL.orders`AS O
Join `scaler-dsml-sql-priyanka.Target_SQL.order_items`as OI
ON OI.order_id = O.order_id
Join `scaler-dsml-sql-priyanka.Target_SQL.sellers`AS S
ON S.seller_id =OI.seller_id
```

```
    WHERE
        O.order_delivered_customer_date IS NOT NULL
        AND O.order_estimated_delivery_date IS NOT NULL
  Group by S.seller_state
      )
  Select delivery_Speed,seller_state,
  Case
  When delivery_Speed < 0 THEN 'Fast'
  Else 'Slow' End as delivery_Speed_Rank
  From State_delivery_Speed
  Group by delivery_Speed,seller_state
  Order by  delivery_Speed_Rank ASC
LIMIT 10;
```

Query Output: -

Query results

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS

Row	delivery_Speed	seller_state	delivery_Speed_Rank
1	9.33	AM	Slow
2	-10.38	SP	Fast
3	-10.5	PA	Fast
4	-10.5	MA	Fast
5	-11.59	RJ	Fast
6	-11.86	BA	Fast
7	-12.25	DF	Fast
8	-12.43	ES	Fast
9	-12.47	CE	Fast
10	-12.53	MG	Fast

Insights: - conclusion is that this Query provides the average difference between actual delivery and estimated delivery,for each state. Whether the delivery is "Fast" (if the average delivery speed is negative) or "Slow" (if it's positive or zero). top 5 states where the order delivery is fast.

6.Analysis based on the payments:

Find the month on month no. of orders placed using different payment types.

Query:-

```
    WITH Order_Counts AS (
    SELECT
    P.payment_type,
```

```
EXTRACT(YEAR FROM O.order_purchase_timestamp) AS Payment_Year,
EXTRACT(MONTH FROM O.order_purchase_timestamp) AS Payment_month,
COUNT(DISTINCT O.order_id) AS Total_Orders
FROM `scaler-dsml-sql-priyanka.Target_SQL.payments`  AS P
JOIN `scaler-dsml-sql-priyanka.Target_SQL.orders` AS O
ON P.order_id = O.order_id
GROUP BY P.payment_type,Payment_Year,Payment_month),
Month_Month_Comparison AS (
SELECT
payment_type,Payment_Year,Payment_month,Total_Orders,
LAG(Total_Orders) OVER (PARTITION BY payment_type ORDER BY Payment_Year,
Payment_month DESC) AS previous_month_count,
LEAD(Total_Orders) OVER (PARTITION BY payment_type ORDER BY Payment_Year,
Payment_month DESC ) AS next_month_count
FROM Order_Counts)
SELECT
payment_type,Payment_Year,Payment_month,Total_Orders,previous_month_count,next_m
onth_count,
(COALESCE(next_month_count, 0) - COALESCE(previous_month_count, 0))
/NULLIF(COALESCE(previous_month_count, 0), 0) AS Month_increase
FROM Month_Month_Comparison
ORDER BY Payment_Year,Payment_month,payment_type DESC
```

Query Output: -

Query results

SAVE RESULTS EX

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	payment_type	Payment_Year	Payment_month	Total_Orders	previous_month_cou	next_month_count	Month_increase
1	credit_card	2016	9	3	253	4363	16.24505928853...
2	voucher	2016	10	11	null	220	null
3	debit_card	2016	10	2	null	64	null
4	credit_card	2016	10	253	1	3	2.0
5	UPI	2016	10	63	null	1160	null
6	credit_card	2016	12	1	null	253	null
7	voucher	2017	1	33	69	4	-0.94202898550...
8	debit_card	2017	1	9	13	277	20.30769230769...
9	credit_card	2017	1	582	1347	4963	2.684484038604...
10	UPI	2017	1	197	398	1139	1.861809045226...

Insights: - conclusion is that this Query provides positive and negative month-over-month changes for different payment types, For instance, for credit_card in 2016-9, the Month_increase is 16.25%

2.Find the no. of orders placed on the basis of the payment installments that have been paid.

```
SELECT P.payment_type,P.payment_installments,
Round(SUM(OI.price),2)as Toatl_price,
Count(Distinct P.order_id)as Total_order_id,
FROM `scaler-dsml-sql-priyanka.Target_SQL.payments` As P
Join `scaler-dsml-sql-priyanka.Target_SQL.order_items`as OI
ON P.order_id =OI.order_id
Group by P.payment_type,P.payment_installments
```

Query Output: -

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	payment_type	payment_installment	Toatl_price	Total_order_id
1	credit_card	0	129.28	2
2	voucher	1	659473.64	3766
3	credit_card	1	2107865.65	25220
4	debit_card	1	183758.74	1521
5	UPI	1	2391525.66	19614
6	credit_card	2	1342053.05	12317
7	credit_card	3	1273163.21	10385
8	credit_card	4	1005395.35	7040
9	credit_card	5	838244.54	5195
10	credit_card	6	721787.06	3895
11	credit_card	7	265336.11	1612
12	credit_card	8	1202462.49	4239

Results per page: 50 1 – 27 of 27

Insights: - conclusion is that this Query provides the total price and the number of orders based on different payment types and payment instalments The majority of orders were paid in “1” instalment, with credit card being the most popular payment method, accounting for the highest total price and order count.

- **Order Trends**: Orders have steadily increased, with a significant rise of 44,772 orders from 2016 to 2017, and a smaller increase in 2018.
- **Seasonality**: Most orders are placed in summer (Jan-Mar), with a peak in the afternoon (13-18 hrs).
- **Regional Distribution**: Orders are spread across 4,119 cities and 27 states, with varying customer densities per state.
- **Order Cost Growth**: The total order cost in 2018 has increased by 136.98% compared to 2017.
- **Freight and Delivery Time**: Analysis shows significant variations in delivery speeds across states, with some states experiencing faster deliveries than estimated.